Greedy Algorithms

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

INTRODUCTION

- **Key Question**
  - ✓ "Can we do better?"

- **How to solve a problem**
  - ✓ Divide and conquer
    - Recurrence equation
  - ✓ Dynamic programming
    - To avoid the repeated same calculation
    - Optimal substructure & Overlapping subproblems
    - Examples
      - » Fibonacci Numbers
      - » Matrix-chain multiplication (Parenthesization)
      - » Longest common subsequences

√ Can we do better?

Growth of functions
$\Theta(\cdot)$, $O(\cdot)$ ⋯
to evaluate (time) complexity

Dynamic Programming is Blind

- **The algorithms we have studied are still <u>relatively complex</u>:**
  - ✓ Matrix chain multiplication (MCM): $O(n^3)$
  - ✓ Longest common subsequence (LCS): $O(mn)$

- **Why?**
  - ✓ We still have many choices to consider for an optimal solution.

$i = 3,\ j = 6,\ k = 3, 4, 5$

$$\min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1}p_kp_j)$$

$k = 3 : m[3,3] + m[4,6] + p_2p_3p_6$

$k = 4 : m[3,4] + m[5,6] + p_2p_4p_6$

$k = 5 : m[3,5] + m[6,6] + p_2p_5p_6$

	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

$m[1..n, 1..n]$

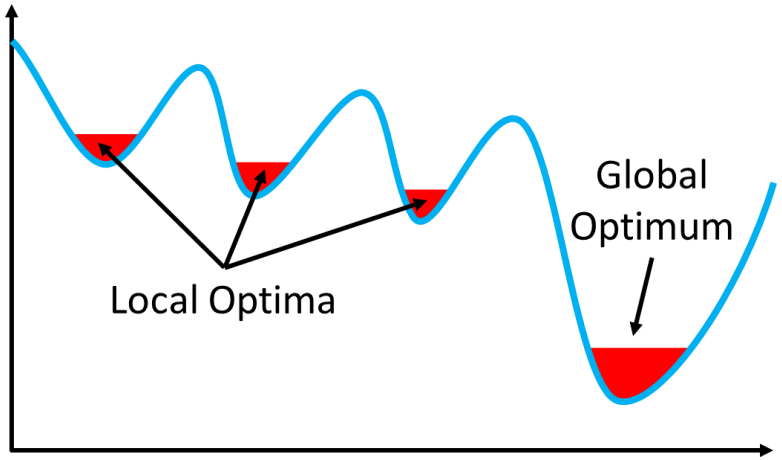
		0	1 A	2 B	3 C	4 B	5 D	6 A	7 B
0		0	0	0	0	0	0	0	0
1	B	0	0	1	1	1	1	1	1
2	D	0	0	1	1	1	2	2	2
3	C	0	0	1	2	2	2	2	2
4	A	0	1	1	2	2	2	3	3
5	B	0	1	2	2	3	3	3	4
6	A	0	1	2	2	3	3	4	4

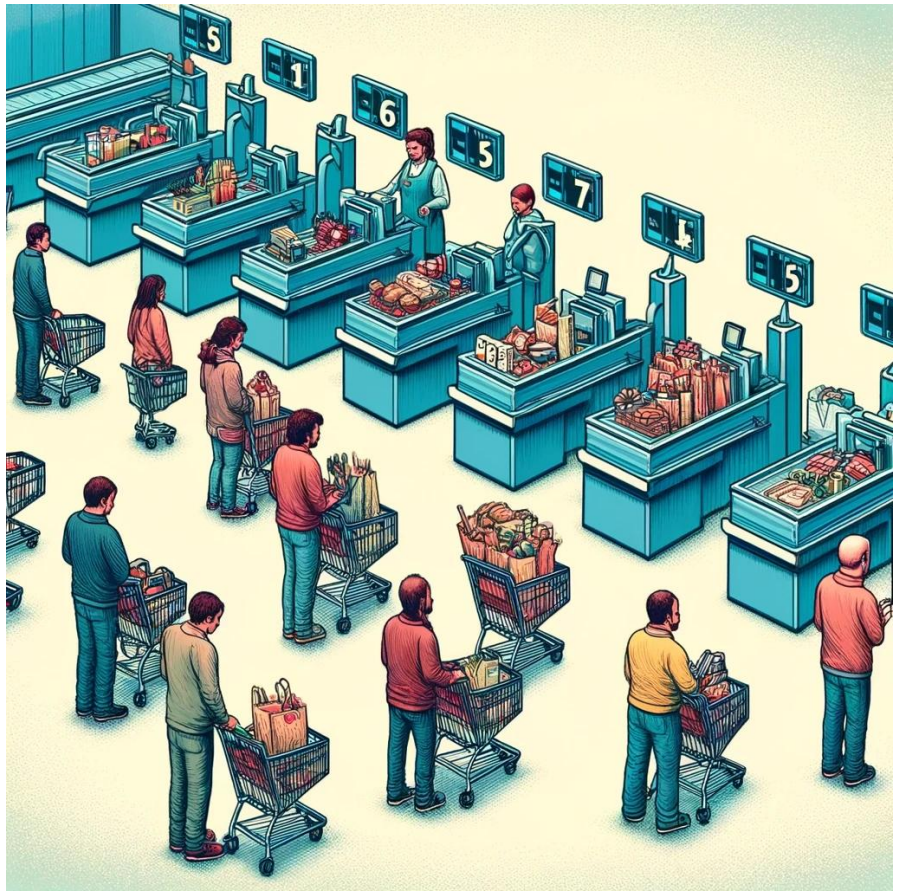
$$c_{ij} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c_{i-1,j-1} + 1 & \text{if } i,j > 0 \text{ and } x_i = y_j \\ \max(c_{i,j-1}, c_{i-1,j}) & \text{if } i,j > 0 \text{ and } x_i \neq y_j \end{cases}$$

GREEDY ALGORITHMS

❖ Greedy algorithm

- ✓ is a design strategy for <u>some</u> optimization problems
- ✓ makes the choice that looks best at the moment without considering the future consequences.
- ✓ does not always lead to a globally optimum solutions.
  - This choice could evoke a local optimum issue.
- ✓ but comes up with the global optimum in <u>many cases</u>.





❖ Greedy algorithm

It is "greedy" because it tries to grab as much as it can in the short term, without thinking about the long-term consequences.

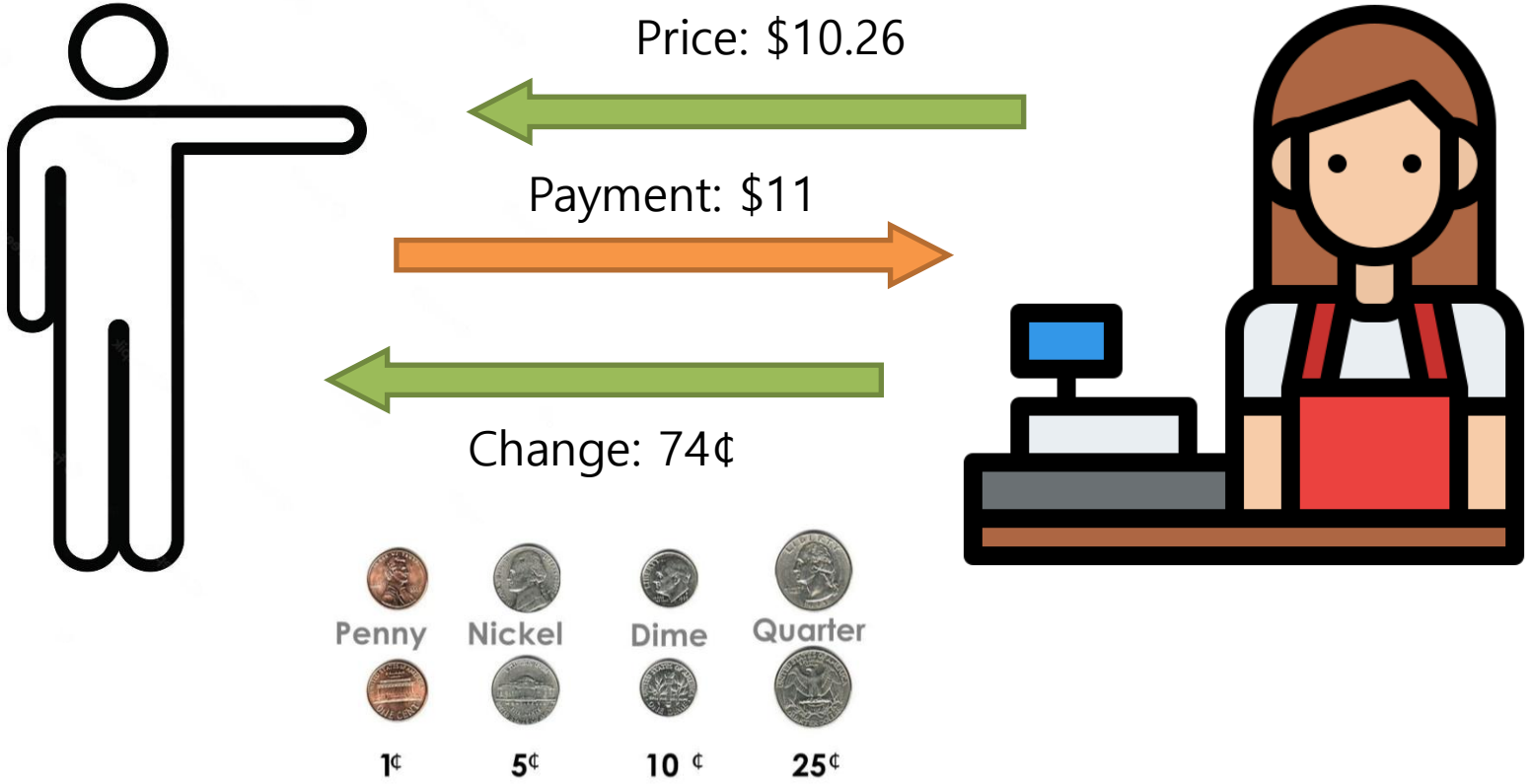
Img: https://www.crosswalk.com/family/parenting/kids/4-important-tips-to-raise-generous-kids-in-a-greedy-world.html

Greedy Algorithm Example

Coin Change

COIN CHANGE

- **Problem:**
  - ✓ We want to make <u>changes with minimum number of coins</u>.
  - ✓ We can solve this problem by using greedy algorithm.



COIN CHANGE

- **Problem:**
  - ✓ We want to make <u>changes with minimum number of coins</u>.
  - ✓ We can solve this problem by using greedy algorithm
- **Algorithm:**
  - ✓ use as **many** coins as possible of the (**next) largest** denomination and so forth.
- **Example:**
  - ✓ coins={50¢, 25¢, 10¢, 5¢, 1¢}
  - ✓ 74¢ =
    - 1(50¢)
    - +2(10¢)
    - +4(1¢)

"Grab as much as it can in the short term without considering further consequences"

Denomination: 액면가

Greedy Algorithm Example

Activity-Selection Problem

ACTIVITY-SELECTION PROBLEM

- **Problem:**
  - ✓ Make the most valuable use of your ticket at the amusement park.
    - There are lots of rides and each of them stars and ends at different times.
  - ✓ Your goal: **ride as many rides as possible**
    - ※ Note
      Another possible alternative goal is "to maximize time spent on rides".
      But we do not solve it here.

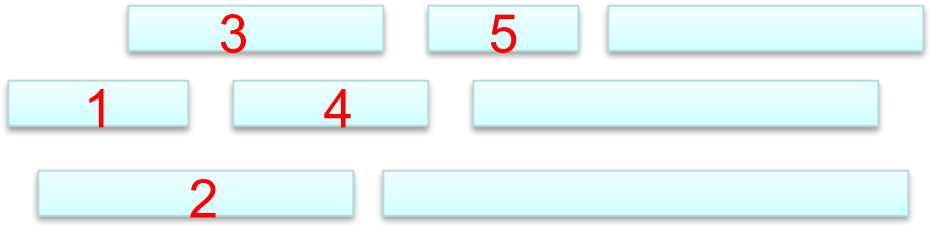
AN ACTIVITY-SELECTION PROBLEM

- **Maximum-size set of mutually compatible activities**
  - ✓ We want to schedule several activities that require exclusive use with a goal of selecting a maximum-size set of mutually compatible activities.

Mutually compatible activities:

AN ACTIVITY-SELECTION PROBLEM

- **Maximum-size set of mutually compatible activities**
  - ✓ We want to schedule several activities that require exclusive use with a goal of selecting a maximum-size set of mutually compatible activities.
- An activity a_i
  - ✓ is represented by a time range $[s_i, f_i)$, where s_i is a start time and f_i is a finish time.
- a_i and a_j are compatible
  - ✓ if $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap.
- $f_1 < f_2 < \cdots < f_n$
  - ✓ Assume that the activities are sorted(indexed) according to increasing finish times, that is, $f_1 < f_2 < \cdots < f_n$.
  - ✓ Thus, $f_i < f_j$ if $i < j$ for a_i and a_j.



AN ACTIVITY-SELECTION PROBLEM

- To select a maximum-size subset of mutually compatible activities.
- **Example:**

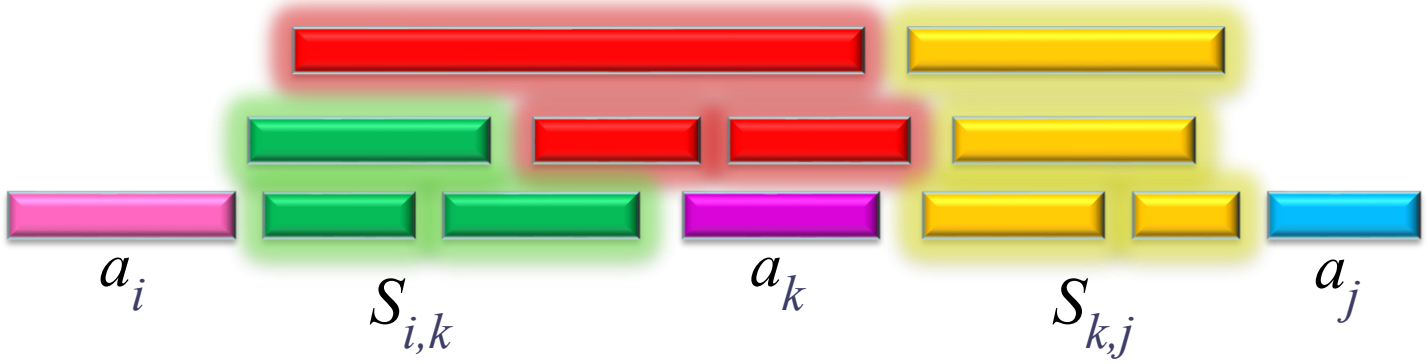
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

Mutually compatible activities: $\{a_{3,}a_9, a_{11}\}$, $\{a_{1,}a_4, a_8, a_{11}\}$, $\{a_{2,}a_4, a_9, a_{11}\}$

Largest mutually compatible activities: $\{a_{1,}a_4, a_8, a_{11}\}$, $\{a_{2,}a_4, a_9, a_{11}\}$

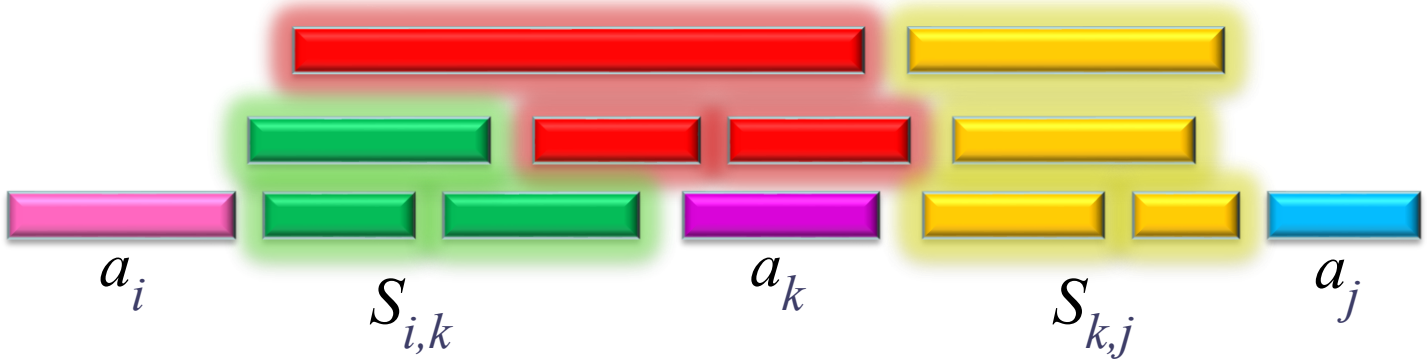
THE STRUCTURE OF AN OPTIMAL SCHEDULE

- Suppose we have a set S of n proposed activities.
- Let $S_{i,j}$ be the subset of activities in S
  - ✓ that <u>begin after</u> activity a_i finishes,
  - ✓ and <u>end before</u> activity a_j starts.
- We can add two fictitious activities a_0 and a_{n+1} with $f_0 = 0$ and $s_{n+1} = +\infty$.
- Then $S_{0,n+1}$ is the set of all activities S.



The Structure of an Optimal Schedule

- Now, it is the time to decrease the size of problem
  - ✓ from $S_{i,j}$ (initially, $S_{0,n+1}$) to $S_{i,k}$ and $S_{k,j}$ where $0 < i < k < j < n$.
- Assume that
  activity a_k is a part of an optimal schedule of the activities in $S_{i,j}$.
- Then, the optimal schedule consists of
  - ✓ an optimal schedule of the activities in a maximal subset of $S_{i,k}$,
  - ✓ $\{a_k\}$,
  - ✓ and an optimal schedule of the activities in a maximal subset of $S_{k,j}$.



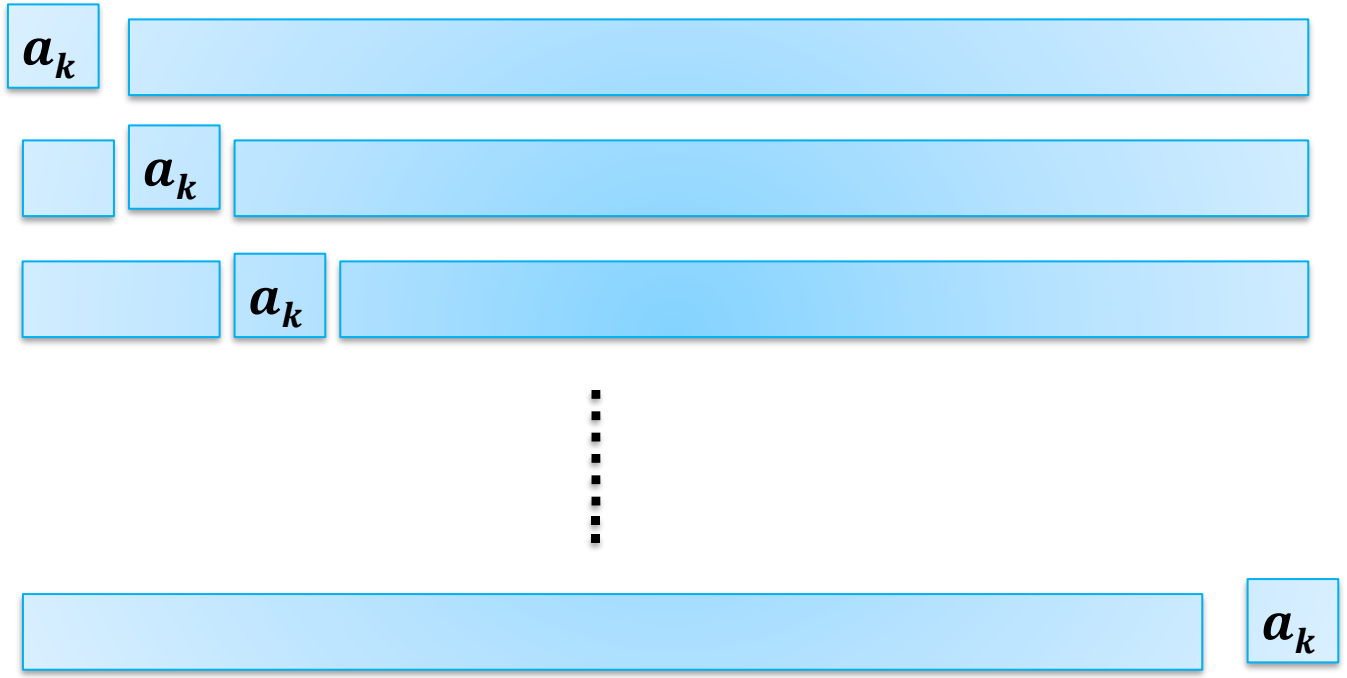
THE STRUCTURE OF AN OPTIMAL SCHEDULE

- Hence, if $c_{i,j}$ is the size of an optimal schedule for set $S_{i,j}$, we have

$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \Phi \\ \max\limits_{\substack{i<k<j \\ a_k \in S_{i,j}}} \{c_{i,k} + c_{k,j} + 1\} & \text{if} \quad S_{i,j} \neq \Phi \end{cases}$$

$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \emptyset \\ \max\limits_{\substack{i<k<j \\ a_k \in S_{i,j}}} \{c_{i,k} + c_{k,j} + 1\} & \text{if} \quad S_{i,j} \neq \emptyset \end{cases}$$

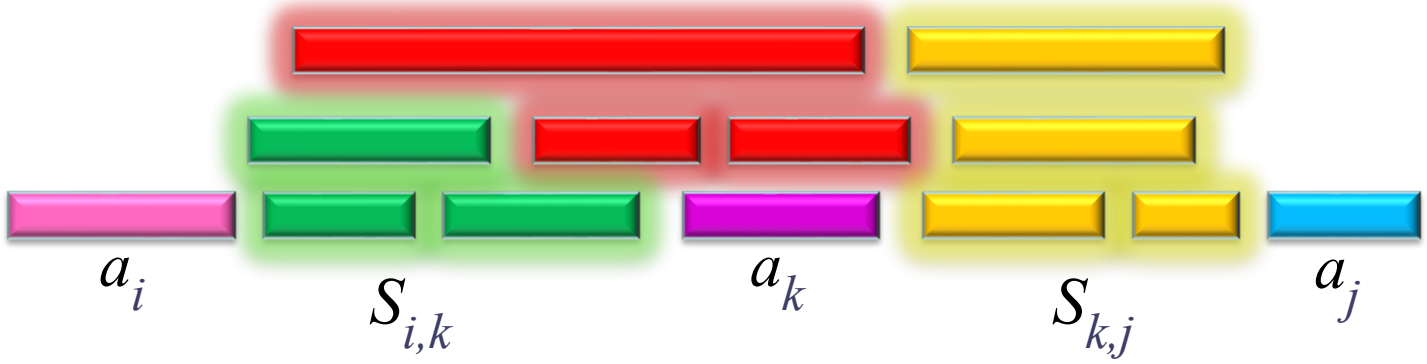
Dynamic Programming



THEOREM

- Let $S_{i,j} \neq \emptyset$,
  and let a_m be the activity in $S_{i,j}$ with the earliest finish time.
  Then,

1. a_m is used in
   some maximum-size subset of mutually compatible activities of $S_{i,j}$
2. $S_{i,m} = \emptyset$,
   so that choosing a_m leaves $S_{m,j}$ as the only nonempty subproblem.

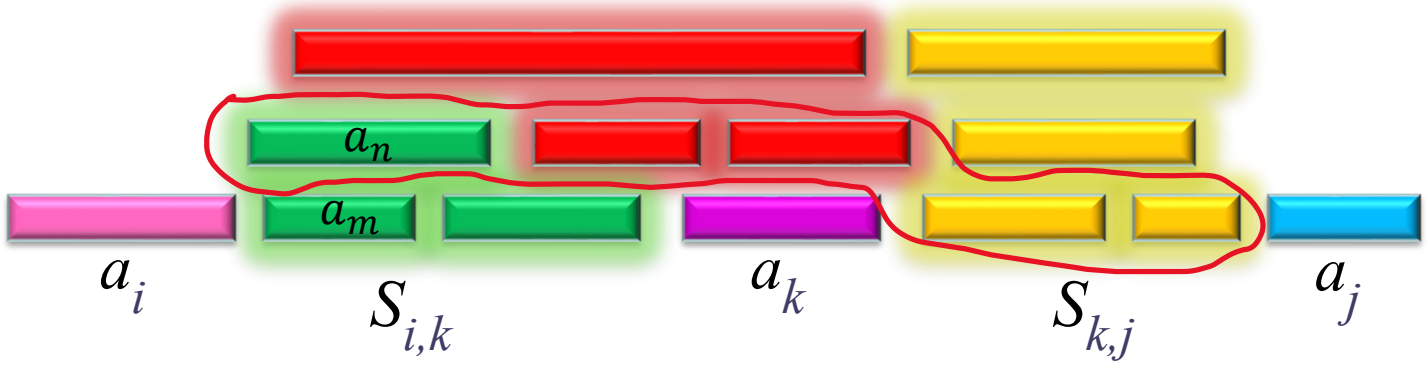


THEOREM

- Let $S_{i,j} \neq \emptyset$,
  and let a_m be the activity in $S_{i,j}$ with the earliest finish time.
  Then,

1. a_m is used in
   some maximum-size subset of mutually compatible activities of $S_{i,j}$

   ✓ **Proof**
   - Let $A_{i,j}$ be a maximum-size subset of mutually compatible activities in $S_{i,j}$.
   - Order the activities in $A_{i,j}$ in monotonically increasing order of finish time.
   - Let a_n be the first activity in $A_{i,j}$.
   - If $a_n = a_m$, done.
   - If $a_n \neq a_m$, replace a_n with a_m to construct $A_{i,j}{}'$, i.e. $A_{i,j}{}' = (A_{i,j} - \{a_n\}) \cup \{a_m\}$.
     $A_{i,j}{}'$ is also a maximum-size subset of mutually compatible activities of $S_{i,j}$ with a_m.



THEOREM

- Let $S_{i,j} \neq \emptyset$,
  and let a_m be the activity in $S_{i,j}$ with the earliest finish time.
  Then,

1. a_m is used in
   some maximum-size subset of mutually compatible activities of $S_{i,j}$
2. $S_{i,m} = \emptyset$,
   so that choosing a_m leaves $S_{m,j}$ as the only nonempty subproblem.

- **<u>Choose the activity in $S_{i,j}$ with the earliest finish time!</u>**

$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \emptyset \\ c_{m,j} + 1 & \text{if} \quad S_{i,j} \neq \emptyset \end{cases}$$

where a_m be the activity in $S_{i,j}$
with the earliest finish time

$c_{i,j} =$ $= \Phi$

$\neq \Phi$

- **Example:**

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

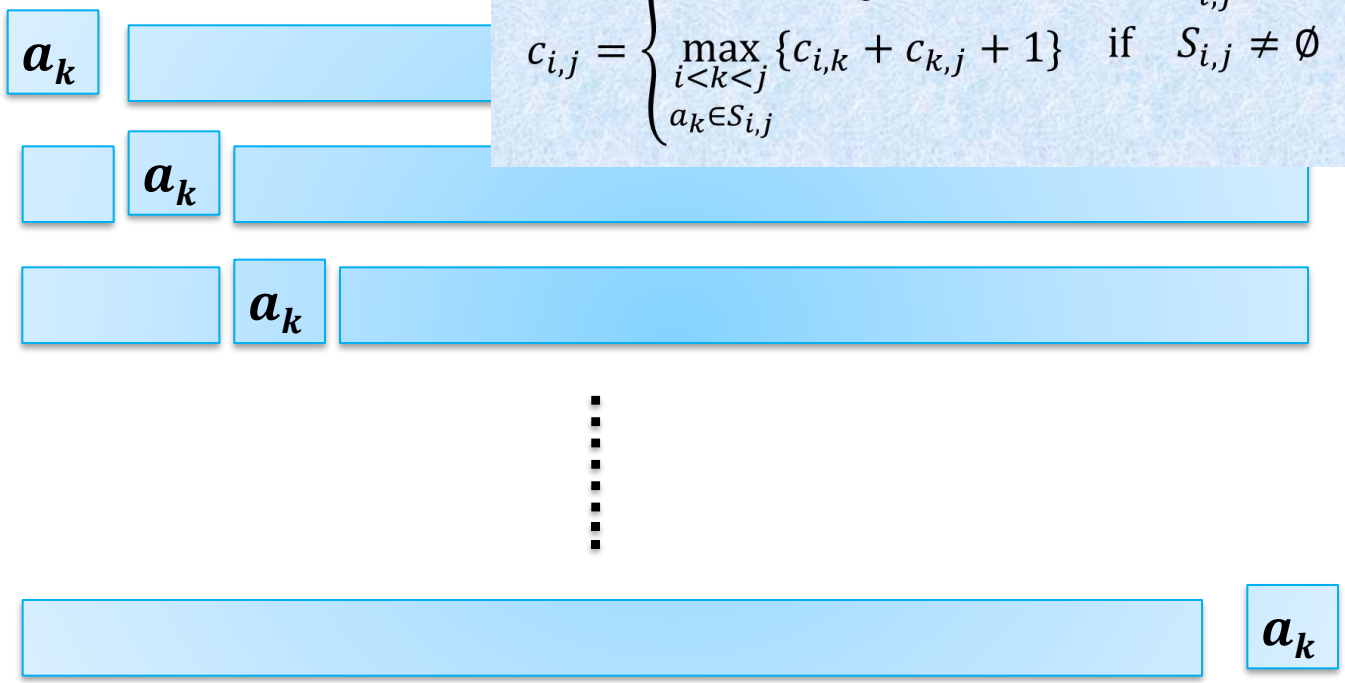
$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \emptyset \\ c_{m,j} + 1 & \text{if} \quad S_{i,j} \neq \emptyset \end{cases}$$

where a_m be the activity in $S_{i,j}$ with the earliest finish time

Dynamic Programming vs. Greedy

$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \emptyset \\ \max\limits_{\substack{i<k<j \\ a_k \in S_{i,j}}} \{c_{i,k} + c_{k,j} + 1\} & \text{if} \quad S_{i,j} \neq \emptyset \end{cases}$$

Dynamic Programming



Greedy

$$c_{i,j} = \begin{cases} 0 & \text{if} \quad S_{i,j} = \emptyset \\ c_{m,j} + 1 & \text{if} \quad S_{i,j} \neq \emptyset \end{cases}$$

where a_m be the activity in $S_{i,j}$ with the earliest finish time

A Variation of the Problem

- If we want to maximize the total time spent on activities in use,
  - ✓ our dynamic programming approach still can be applied.

- **None** of the following greedy choices would work:
  - ✓ Choose the activity that starts earliest/latest
  - ✓ Choose the activity that finishes latest
  - ✓ Choose the longest activity

Thank you!